

Nociones básicas de testing

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas - M Victoria Tolosa

SOBRE BUGS



Bug

Al comenzar las clases comentamos el concepto de bug. Básicamente es un mal funcionamiento del código.

El mismo puede ser por errores:

- Sintácticos
- De ejecución
- Semánticos

Siendo estos últimos los más peligrosos y difíciles de encontrar.

Además de esto, existen las **fallas técnicas**.

Bugs/fallos famosos

A lo largo de la Historia ha habido varios bugs famosos, pero hagamos historia reciente:

- Famoso caso de bugs sintácticos: Cyberpunk 2077 (2020)
- Famoso caso de fallas de tecnología: Episodio AWS Cono Sur (12/12/2025)
- Famoso caso de bug de ejecución: Aterrizaje del Apolo 11

Cyberpunk 2077



- Famoso videojuego con un gran fanbase
- Su lanzamiento estuvo lleno de glitches (bugs de sintaxis que literalmente “traban” el juego.
- Millones de dólares de pérdida.
- Hasta el día de hoy reporta bugs (mucho menos)

Incidente de AWS Cono Sur 12/12/2025



- Los servidores de AWS tuvieron una sobrecarga de tráfico, además de errores en actualizaciones de APIs.
- Tuvieron que reparar servidores físicos en data centers.
- Esto implicó un día entero con el servicio caído.
- La principal billetera virtual del país, por ende, tampoco funcionó
- Millones de dólares de pérdidas

Apollo 11



- Misión espacial estadounidense.
- El código procesaba más datos de los que debía, saturando la CPU y forzando alunizaje manual.
- Podría haber significado una tragedia humana (además de haberse perdido **muchos** millones de dólares).
- Por suerte todo salió bien.

La certeza no existe (en la práctica)

Mediante buenas prácticas y testing uno puede “ganar confianza” con el código que crea, pero: es posible estar 100% seguro de que un script no generará bugs?

Teóricamente, **si**.

- Primero se hace una especificación formal del programa.
- Luego se demuestra su factibilidad para cualquier tipo de input.

En la práctica **nadie hace esto** ya que es increíblemente tedioso y no va de la mano con los tiempos lógicos de producción.

Tipos de bugs de sintaxis

Algunas situaciones frecuentes:

- Error de indentación.
- Faltan : luego de guarda del condicional o ciclo.
- Usar = en lugar de == en una guarda.
- Desbalance de expresiones con () o .
- Nomenclatura de variables con palabras reservadas.

Son los más fáciles de corregir. La mayoría de las veces el mismo lenguaje te indica cuál es el error.

Tipos de bugs de ejecución

Algunas situaciones frecuentes:

- Ciclos infinitos.
- Recursión infinita.
- El flujo de ejecución no es el correcto.
- Excepciones encontradas por Python durante la ejecución.

Como su nombre lo indica, suelen aparecer en medio de una ejecución. El script “comienza a correr” sin detectar errores de sintaxis y a la mitad de la ejecución “algo sucede”.

Tipos de bugs de ejecución

Las excepciones más comunes:

- **NameError**. Estamos tratando de usar una variable que no está definida en el entorno actual.
- **TypeError**. Estamos tratando de operar con un valor de manera incorrecta, o pasando mal los parámetros a una función, entre otros.
- **KeyError**. Queremos acceder a un elemento de un diccionario con una clave no definida.
- **IndexError**. Estamos accediendo a una posición inválida de una lista, string, tupla.

SOBRE TESTING



Testing: intuición

Como su nombre lo indica, testear un código es básicamente someterlo a una serie de **casos límite** para evaluar su comportamiento.

Ejemplo: tenemos una function que se encarga de extraer datos y guardarlos en una base de datos.

- Qué pasa si el volumen de datos a extraer es ENORME?
- Qué pasa si vienen datos duplicados?
- Qué pasa si al extraer no viene ningún dato?

Está bueno generar un test unitario para cada uno de estos casos border

Bugs semánticos

Como ya dijimos, son los más difíciles de detectar. Se dan cuando el script ejecuta bien pero no da los resultados deseados.

Vemos que estos son los tipos de bugs sobre los cuáles tiene sentido realizar tests unitarios.

Por qué no tiene sentido realizar tests sobre errores sintácticos o de ejecución?

Lógica

La lógica de un test unitario es plantear casos en los que sabemos el resultado: si la function efectivamente hace lo que queremos, este test tiene que dar X resultado.

Para hacer tests unitarios, en esta materia usaremos la **librería unittest**.

EJEMPLO



Ejemplo

Decimos que dos string son similares si:

- Tienen la misma longitud.
- La cantidad de posiciones con caracteres diferentes es a lo sumo 2.

Formulamos algunos ejemplos (los casos de test) para los cuales conocemos el resultado:

s1	s2	resultado esperado
casa	casa	True
casa	csad	False
csas	caas	True
bbbb	casa	False

Ejemplo

Resolvemos así:

```
def son_similares(s1,s2) :  
    if len(s1) != len ( s2 ) :  
        return False  
    cnt = 0  
    for i in range(0,len(s1)) :  
        if s1[i]!= s2[i]:  
            cnt = cnt + 1  
    return (cnt <= 2)
```

Esperamos esto:

s1	s2	resultado esperado	son_similares_v1(s1,s2)
casa	casa	True	?
casa	csad	False	?
csas	caas	True	?
bbbb	casa	False	?

Cómo quedarían los tests de esta function usando la librería unittest?

Ejemplo

```
import unittest
from similares import *

class TestSimilares(unittest.TestCase):
    def test_similares (self) :
        self.assertTrue(son_similares( 'casa' , 'casa' ))
        self.assertTrue(son_similares( 'csas' , 'caas' ))
        self.assertFalse(son_similares( 'casa' , 'csad' ))
        self.assertFalse(son_similares( 'casa' , 'bbbb' ))
        self.assertFalse(son_similares( 'casaaa' , 'bbbb' ))
        self.assertFalse(son_similares( 'casaaa' , 'bbsaab' ))

if __name__ == '__main__' :
    unittest.main ( )
```

Ejemplo

Nomenclatura:

- Los tests deben definirse en un archivo aparte, importando la librería unittest e importando la/s functions a testear desde otros scripts (debe estar todo en la misma carpeta).
- Se define una **clase**. La misma puede llamarse de cualquier manera pero su argumento debe ser **unittest.TestCase**
- Dentro se define una function por cada set de tests que queramos tener. El argumento de dicha function es **self**.
- Dentro de esta function se definen los tests puntuales.
- Todo se llama con un `unittest.main()`

Ejemplo

Justamente esta function está bien formulada, por lo tanto el resultado será que todos los tests

pasaron la prueba

```
In [21]: unittest.main ( )
```

```
.
```

```
-----  
Ran 1 test in 0.000s
```

```
OK
```

Qué pasa si modificamos ahora la function de manera de que haya pruebas que fallen?

```
def son_similares_falla(s1,s2) :  
    if len(s1) != len ( s2 ) :  
        return False  
    cnt = 0  
    for i in range(2,len(s1)) :  
        if s1[i] != s2[i]:  
            cnt = cnt + 1  
    return (cnt <= 2)
```

Ejemplo

```
import unittest
from similares import *

class TestSimilares(unittest.TestCase):
    def test_similares (self) :
        self.assertTrue(son_similares( 'casa' , 'casa' ))
        self.assertTrue(son_similares( 'csas' , 'caas' ))
        self.assertFalse(son_similares( 'casa' , 'csad' ))
        self.assertFalse(son_similares( 'casa' , 'bbbb' ))
        self.assertFalse(son_similares( 'casaaa' , 'bbbb' ))
        self.assertFalse(son_similares( 'casaaa' , 'bbsaab' ))

    def test_similares_2 (self) :
        self.assertTrue(son_similares_falla( 'casa' , 'casa' ))
        self.assertTrue(son_similares_falla( 'csas' , 'caas' ))
        self.assertFalse(son_similares_falla( 'casa' , 'csad' ))
        self.assertFalse(son_similares_falla( 'casa' , 'bbbb' ))
        self.assertFalse(son_similares_falla( 'casaaa' , 'bbbb' ))
        self.assertFalse(son_similares_falla( 'casaaa' , 'bbsaab' ))

if __name__ == '__main__' :
    unittest.main ( )
```

Ejemplo

```
In [26]: unittest.main ( )
.F
=====
FAIL: test_similares_2 (__main__.TestSimilares.test_similares_2)
-----
Traceback (most recent call last):
  File "/home/pablo/Escritorio/coso_unittest_clase/tests.py", line 17, in
test_similares_2
    self.assertFalse(son_similares_falla( 'casa' , 'csad' ))
AssertionError: True is not false
-----
Ran 2 tests in 0.001s

FAILED (failures=1)
```

Vemos que el test te especifica: la cantidad de failures, cuáles fueron los tests particulares que fallaron y en qué function están.

En la vida real

Hay muchas maneras de testear:

- Testeos automáticos (CI/CD)
- Code Review (uso de repositorios)
- Test Driven Development (no tan común).

De tarea

Crear tests unitarios para cada uno e los ejercicios del TP1